

What's in the middle?

S P Bennis, B A F Crawford and M Touman

Enterprise Middleware has the power to help you quickly develop, deploy and manage critical on-line transaction processing and client/server applications to take advantage more easily of new business opportunities and to manage, change or extend your business — but first you need to know what to look for in the increasingly diverse range of technology products. This paper discusses the issues that a programme in BT should consider when selecting middleware products to deploy with BT systems. The paper draws upon the experiences gained within the Client/Server Evolution Laboratory (CSLab), part of the Ipswich Software Engineering Centre's (ISEC's) Systems and Software Strategy Unit, with particular emphasis on the Object Management Group's Common Object Request Broker Architecture (CORBA), Distributed Computing Environment and Java.

1. Introduction

Enterprise Middleware and its role within BT's operational support systems (OSS) is the subject of much discussion and conjecture. It is widely agreed that there is a need within BT for a consolidated set of middleware products to enable BT developers to rapidly produce mission-critical OSS in a consistent manner, with minimal product diversity and maximum ability to manage that diversity. This paper aims to give an insight into the world of distributed computing, the different approaches available, the kind of issues that BT needs to consider for its OSS systems, and an overview of four possible solutions (CORBA, DCE, DCOM and Java RMI).

2. History of distributed systems

2.1 Monolithic systems and mainframes

At the dawn of the computing age came the mainframe. Mainframes usually cost a great deal to maintain but are capable of serving large numbers of users and have the (dis?)advantage of being centrally managed. Software systems were often written to be monolithic, i.e. the user interface, business logic and database access were all contained in one large application. A typical architecture of such a system is illustrated in Fig 1.

2.2 Client/server architecture

The advent of the PC ensured that a dramatic paradigm shift from the monolithic architecture of mainframe-based applications became a practical option. Whereas the monolithic system architecture required the mainframe to carry out all the processing, the advent of the client/server architecture allowed some of the processing to be carried out on the desktop. As a consequence of the reduction of

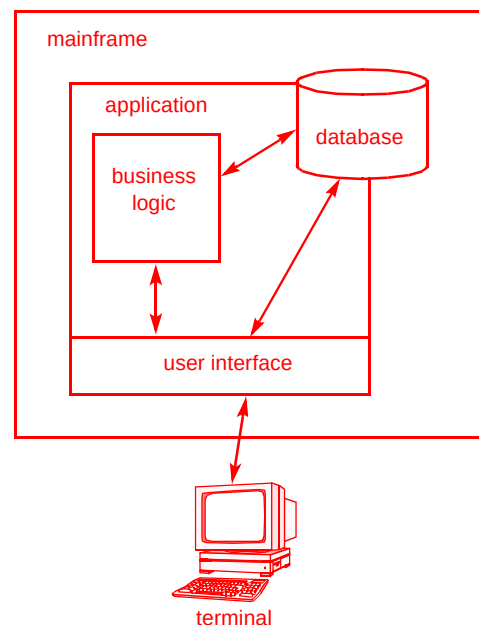


Fig 1 Typical monolithic application architecture.

processing required by the 'central' machine, this architecture also witnessed the proliferation of Unix-based servers, with a lower purchase cost than the mainframe machines they replaced. The use of these cheaper servers also allowed the more widespread deployment and use of servers within smaller teams, rather than a low number of centrally managed mainframes for the whole company.

Client/server applications typically split the application's components such that the database resided on the server, the user interface on the client, and the business logic would reside on either or both. This 'two-tier' split meant that any changes to the client components would

result in new copies of the software application being distributed to all users of the system. A typical architecture for this two-tier system is illustrated in Fig 2.

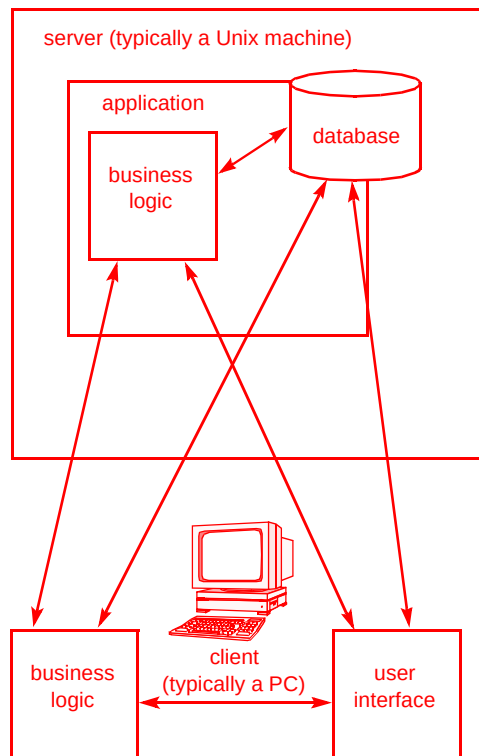


Fig 2 Two-tier client/server architecture.

2.3 Multi-tier client/server

The client/server revolution did solve a number of the problems associated with mainframe systems. However, it did not provide all the answers. With database access functionality and, potentially, business logic residing on the client machines, any changes to either the database structure/access routines or indeed business logic, resulted in a logistical problem of updating all the clients to the new system. The multi-tier client/server model addressed a number of the problems inherent in the two-tier client/server architecture. Conceptually, an application can have any number of tiers, but the most popular multi-tier architecture is three-tier, which partitions the systems into three logical groupings — the user interface layer, the business logic layer and the database access layer. Figure 3 illustrates this architecture.

A multi-tier client/server architecture enhances the two-tier model through the insulation and separation of the layers. Consequently the user interface layer communicates only with the business rules layer, never directly with the database access layer. Therefore any business logic or database-related changes could be carried out without impacting on the clients, as long as the interface between the client and business logic layer remains consistent.

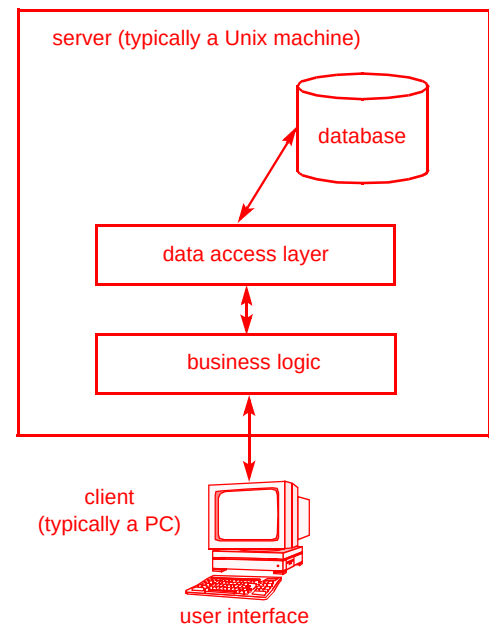


Fig 3 Three-tier client/server architecture.

2.4 Distributed objects

The concept of distributed objects takes the multi-tier architecture to its next evolutionary stage. Where the multi-tier architecture differentiated between business logic and data access, the distributed object architecture exposes the functionality of the whole application as objects, each of which can, in turn, use any of the functionality of the other objects in the system, or even objects in other systems. The concept of 'clients' and 'servers' can also be blurred since the client components can create objects that behave in server-like roles. Hence the application of the object-oriented methodology to an *n*-tier environment results in distributed component-based systems.

3. Distributed computing approaches

When designing and implementing distributed applications, there are a number of alternative mechanisms for a developer to consider to support remote interaction between software components. The choice made will be dependent upon the nature of the application, ranging from its complexity to the platform(s) on which it runs and the language(s) used to implement it. The hierarchy of distributed computing approaches is shown in Fig 4, with a brief explanation of each being given in the following sections.

3.1 Socket programming

In most modern software systems, communication between machines, and sometimes between processes in the same machine, is done through the use of 'sockets'. A socket is a channel by means of which applications can connect to each other and communicate. Socket

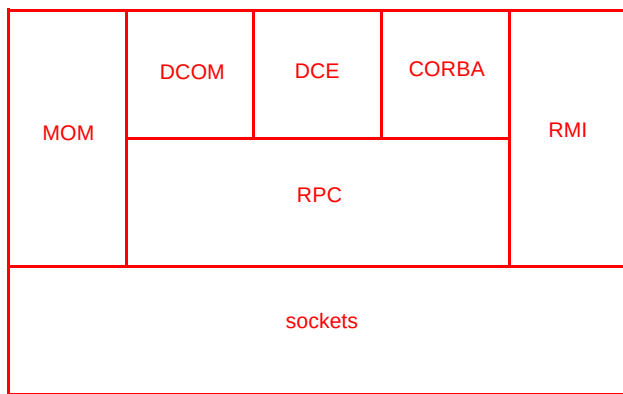


Fig 4 Hierarchy of distributed computing approaches.

programming is the use of sockets to communicate between applications by writing data to and/or reading data from a socket. The application programming interface (API) for socket programming is rather low-level and as a result the overhead associated with an application that communicates in this fashion is very low. On the negative side, however, there is a lack of type safety and tool support for socket programming leading to longer development times and a greater testing overhead. In addition, complex data types, or data transfer across different machines or languages, are not easily handled by socket programming. Therefore, although socket programming can result in efficient applications, this approach is not suitable for complex applications.

3.2 Remote procedure call (RPC)

The next rung up the distributed applications development ladder from socket programming is the 'remote procedure call'. Using RPC, rather than directly manipulating the data that flows to and from a socket, the developer defines a function and, using a code generator, generates code that makes that function look like a local procedure to a remote caller. In reality the function uses sockets to communicate with the remote server which, upon completion, uses sockets to return the result. This procedural interface makes RPC easier to use than raw socket programming.

RPC programming is the base upon which a number of higher level programming models have been built.

3.2.1 DCE

The Distributed Computing Environment (DCE) is an industry-standard, vendor-neutral set of distributed computing technologies pioneered by the Open Group (formerly known as the Open Software Foundation). It provides security services to protect and control access to RPC interfaces, name services that make it easy to find distributed resources, and a highly scalable model based on a standard RPC for organising widely scattered users, services, and data. DCE runs on all major computing

platforms and is designed to support distributed applications in heterogeneous hardware environments.

3.2.2 CORBA

The Common Object Request Broker Architecture (CORBA) is a vendor-neutral standard for middleware. It is managed by the Object Management Group (OMG), who operate a well-defined process for achieving agreement across its 800+ members — these members include all the major vendors of middleware and large consumers such as banks and telecommunications companies.

CORBA forms only a part of the overall OMG architecture, which is called OMA — the Object Management Architecture. OMA also includes the CORBAServices, which define standard infrastructure services for distributed systems development, the CORBAFacilities which describe more business-related general services, and domain-specific interfaces for industry sectors such as telecommunications and finance.

Although its standards now address a wide range of aspects, OMG's main focus is still APIs, protocols and services that are required for development of scalable, heterogeneous distributed systems, i.e. CORBA and CORBAServices. For the most part it standardises technology that is already in existence, rather than trying to invent new technology from scratch.

It is very important to distinguish between CORBA as a standard, and middleware products which conform to CORBA (Object Request Brokers — ORBs). As with all standards, CORBA only specifies certain aspects of middleware, so many characteristics are therefore product-specific. In particular, aspects such as performance and resilience are product-specific, and there is little merit in generalising about CORBA-compliant products in these areas.

3.2.3 DCOM

The Distributed Component Object Model (DCOM), Microsoft's entry into the distributed computing market, offers capabilities similar to CORBA. DCOM is a relatively robust object model that enjoys particularly good support on Microsoft operating systems because it is integrated with Windows95 and WindowsNT. DCOM, unlike CORBA, is not a standard and is not aimed at heterogeneity or scalability, and availability outside the realms of the Windows operating system is sparse.

3.2.4 RMI

The Java Remote Method Invocation (RMI) is a very CORBA-like architecture with a few twists. RMI has the advantage that it supports the passing of objects by value,

but has the disadvantage that it is a Java-only solution. As a consequence, RMI may be a good choice for an all-Java application, but severely restricts any future modifications to the system that may require interoperability with applications written in other languages.

3.3 Message-oriented middleware

The fundamental aim of message-oriented middleware (MOM) is to provide a very simple mechanism for asynchronous, high-performance communications between applications. The basic service offered is the ability to send a parcel of data to a known destination and to be able to receive incoming data.

There are two principal views of the MOM world — message passing and message queuing. Both of these have their roles to play in different circumstances.

Message passing, whereby if the destination application is not reachable within a fairly short time, an error condition occurs, is applicable to systems concerned with on-line transaction processing (OLTP). A typical scenario of this type of system is from booking offices to a central reservation system.

Message queuing is where the message system takes the data and places it on a local queue until it can be delivered to the remote system where it is again placed in a queue until an application explicitly removes it. These queues can be in memory but are often backed up in persistent storage on the remote system.

4. What is middleware?

The term 'middleware' is open to a great deal of interpretation. Some consider it to be any software that does not implement business logic, while others associate it more closely with support for distributed computing. In this paper, middleware is considered to include:

- support for production and use of physically distributed systems with appropriate non-functional capabilities (i.e. distributed computing),
- support for integration of systems produced using different platforms, languages, middleware products and technologies, and run by different organisations,
- support for efficient development of business applications, where developer effort required for non-business related aspects is minimised.

In addition, middleware is considered not to include:

- support for data management and data distribution of the kind provided by DBMS products.

5. Middleware issues

A full and complete statement of BT's requirements for such middleware has not yet been produced. In an attempt to plug this gap, the following issues have to be addressed.

- What are the current and likely future commercial pressures and drivers on BT's OSS?
- What support can third-party middleware products provide to enable BT to address these drivers?

The key way in which use of middleware can help to meet commercial requirements is by reducing the amount of non-functional software which BT engineers need to develop, test and maintain. This includes any software that does not directly perform a business-related function, e.g. software to implement remote messaging, server management, security or transaction management.

By providing as many of the facilities that are not directly concerned with BT's core business as possible, use of middleware frees BT engineers to focus on software that has direct business benefit, and enables business benefit to be delivered much more rapidly.

As part of the Enterprise Middleware Initiative, the CSLab has begun the process of identifying the key requirements for Enterprise Middleware in relation to BT's OSS. The following sections give a brief overview of the main areas found within this activity. Further information on any aspect of this study can be found at the CSLab's Web pages [1].

5.1 Commercial drivers on BT's IT systems

The commercial drivers on BT's IT systems are of high importance but are sometimes overlooked during the decision-making process. The issues that have been identified include:

- time to market,
- maintenance costs,
- deployment time,
- dynamic business relationships,
- corporate re-structuring,
- new technology,
- use of third-party OSS components,
- OSS customisation,
- OSS integration across services,
- on-line non-functional characteristics,

- OSS integration across providers,
- dynamic portfolio,
- growth in on-line services market,
- support for sales and marketing.

5.2 *Support for distributed computing*

Historically, this has been the main focus of most middleware products. The need for distributed computing is motivated by:

- non-functional requirements such as high levels of performance, availability and scalability that cannot be met by centralised solutions,
- the need to support interaction between organisations or business units that are geographically remote or administratively distinct from each other, such as customers and their service providers.

This section describes the distributed computing facilities required from Enterprise Middleware.

5.2.1 Remote interaction

The most fundamental requirement for distributed computing is support for remote interaction between component instances. For any form of remote communication, failures such as networking errors should be identified by the middleware and this information made available to applications.

In addition to conventional synchronous RPC semantics, support is required for asynchronous messaging, where the sender does not need to wait for a response, yet delivery can be guaranteed.

For multimedia applications, support is required for efficient exchange of multimedia streams (e.g. voice or video). This should enable use of multicast to reduce bandwidth consumption, and include timing support for synchronisation of streams.

5.2.2 Type management

It is important for developers to be able to trap as many potential errors as possible prior to run-time. Strong typing for access to remote interfaces is an important ingredient of this, since testing is one of the key overheads in distributed systems development.

In addition, middleware type systems should support:

- dynamic typing, where new types of objects and interfaces can be added to the system on-line,
- version management of types, to enable migration and yet prevent mismatches between clients and servers,
- sub-typing of interfaces, to reduce redundancy and improve reuse and maintainability.

5.2.3 Binding

Binding is the process of setting up a connection between a client and a server interface — it is a point at which a client can select which server to connect to. It is through this process that resources such as network connections and buffers are allocated.

To simplify development, developers should not need to be exposed to the binding process at all. However, explicit control over binding should be possible if needed.

Binding facilities should support:

- explicit selection of servers via a number of different parameters,
- the ability for servers to export addressing information for use with multiple network protocols, and for clients to select at will from these,
- distribution of client requests across pools of servers in order to distribute or balance load,
- dynamic activation and deactivation of servers and objects to adjust to system load and save computing resources,
- the ability for well-known interfaces to be exported using constant network addresses,
- multiparty bindings, to support multimedia applications, groupware and multicast protocols.

5.2.4 Directory services

In order for potentially remote software components to find and make contact with each other, some remotely accessible repository of information describing the available interfaces is required.

Directory services generally manage and export such repositories, and they should enable clients to obtain addressing information for servers, either using a single name, or by queries with sets of criteria.

Directory services should support:

- servers that can provide clients with addressing information for connection via multiple network protocols,
- server groups for load distribution and resilience,
- dynamic activation of servers or objects when their references are retrieved by clients,
- use of context-relative names with an arbitrarily complex structure of naming contexts that can be modified on-line,
- replication of directory servers and persistent storage of directories, to support scalability and resilience.

5.2.5 Life-cycle support

One component may often wish to create another — for example, a bank may wish to create a new account. In distributed systems, such components may be best situated in different locations, as dictated by design considerations. This means that the capability for components to create other components remotely is important.

5.2.6 Systems management

Management is one of the most important areas for distributed systems, since, in the long term, system administration costs form a significant component of overall maintenance costs, and the quality of service offered by applications can be greatly compromised by inadequate management facilities.

A number of core management operations, such as activate, de-activate, suspend and resume, should be supported on key entities within distributed systems, such as servers, messages and transactions. It should be possible for the current and historic status and state of these entities to be monitored and queried at will. The relationships between these entities should also be available.

In addition, the following issues are important for middleware management facilities:

- management services should be replicated and distributed for resilience and scalability,
- it should be possible to automate standard management tasks, and have them performed automatically under specified conditions or times,
- management of administrative roles and privileges should fall within well-defined management domains,
- remote software distribution and installation, and associated version management issues, should be from a centralised location.

5.2.7 Security

Security support should enable principals — both human users and software — to be identified and authenticated. Fine-grained control is required over which principals are permitted to access which services and resources, with the lowest level of granularity being individual operations and object attributes.

Unauthorised attempts to access resources such as interfaces should, optionally, be recorded and/or notified to administrators; configuration of how this happens should be possible on-line for each instance and type of resource.

Confidentiality and integrity should be preserved using encryption techniques, but middleware should not be prescriptive about which algorithms are used.

5.2.8 Internet access

Middleware should enable secure, manageable and scalable interconnectivity between intranets and the Internet.

In particular, the intranet security, monitoring and management services provided by middleware should be easy to integrate with third-party Internet firewalls.

5.2.9 Audit and traceability

Middleware should allow the activities of principals and their effects to be recorded and analysed. Information about an activity should include which components carried out the activity, when, and on the authority of which principal. This should support detection and prevention of security violations, and enable non-repudiation.

5.2.10 Transactional integrity

Support for data integrity is critical in large-scale systems where information from many different databases may be involved in a single business process.

Transactional support in middleware should enable arbitrary collections of remote messages and any changes to persistent resources that they cause, to be grouped as transactions, and thus to obey the ACID properties. Middleware should be able to co-ordinate a two-phase commit process across these potentially distributed resources.

Transaction co-ordinators should be able to interwork with transaction co-ordinators of other middleware products and information management systems such as DBMSs. This should enable transactions to span different systems and technologies.

Some transactions within BT systems could potentially be very large and complex, such as the set-up of end-to-end global information services. Transaction nesting should also be supported, so that sub-components of a transaction can be committed independently of other sub-components.

5.2.11 Quality of service

The quality of service of links between clients and servers should be configurable, in terms of issues such as whether delivery is guaranteed or not, speed of delivery, and the priority of the information relative to that of other links.

5.2.12 Problem detection and debugging

As testing is widely held to be the greatest difficulty introduced by the change from centralised to distributed systems, support for problem detection and diagnosis is very important.

Strong typing and version management of interfaces should help reduce the likelihood of problems. However, integrated support for distributed monitoring and debugging is an important middleware requirement.

5.2.13 Performance management

Management of performance involves being able to ascertain the performance of components within the system, and being able to re-assign resources or alter the behaviour of the system on-line to rectify performance and optimise overall efficiency.

To support this, monitoring facilities should enable the throughput of individual services to be queried on-line, along with the throughput and round-trip time for specific interfaces or specific bindings between clients and servers.

5.3 *Support for technology integration*

Support for integration of IT systems based on different hardware, communications and software technologies is becoming increasingly important:

- many corporations now have a wide range of different technologies deployed within their IT systems,
- there is a greater need to open up IT systems for co-operation with partners, as in alliances, mergers and joint ventures,
- there is an increasing drive to buy in third-party packaged solutions, which are often based on different technologies.

This section describes some of the areas where middleware can provide support for technology integration,

both in terms of integration with existing systems, and deployment of new systems which can be more easily integrated with future technologies.

5.3.1 Network protocols

Middleware should not be prescriptive about which network protocols are supported, and ideally it should enable support for new network protocols to be incorporated by developers where required.

5.3.2 Programming languages

Middleware should enable components that are developed using different programming languages to interwork fully, with no restrictions placed on interfaces by language choice.

5.3.3 Interface definition languages (IDLs)

A number of different standards for interface definition languages now exist within the industry. Middleware should support multiple IDLs, and should include support for automatic translation between IDLs where possible.

5.3.4 Platforms

Middleware should support multiple platforms and operating systems, and should provide a consistent set of facilities for application developers across these, to help shield developers from the inconsistencies between them.

5.3.5 Modularity

Middleware should be modular, so that some elements of a middleware product can be used while others are not. The components of a middleware product should thus be designed in such a way that they make minimal assumptions about which other components are to be used and in what ways.

In addition, where industry standard APIs are supported, use of services built on top of these should not prevent direct use of the standard APIs.

5.3.6 Open standards

Middleware should support industry standards such as XA, CORBA, SQL and HTTP, wherever appropriate.

5.3.7 Support for key existing technologies

Middleware products should provide compatibility with important existing products such as CICS, IMS and DB2.

5.4 Support for application development

As well as support for production of distributed, heterogeneous systems, middleware should provide tool support to make the process of application development more effective.

This usually comes in the form of integrated tool support:

- design tools,
- compilers and debuggers,
- performance analysis tools,
- version control systems.

Other key issues include:

- independence of business logic from code concerned with distribution technologies, database technologies and other infrastructure issues,
- support for component-based models such as Microsoft's Component Object Model and Enterprise JavaBeans.

5.5 Non-functional characteristics

Since middleware can have such a large impact on the non-functionals of distributed applications, the non-functional characteristics of middleware products are of great importance, most notably performance, scalability, resilience and reliability.

6. Middleware technologies

Having identified the issues to be addressed by any middleware deployments within BT, this section attempts to identify the potential of three offerings, CORBA, DCE and DCOM. While the latter two are actual products, CORBA is a standard and hence there are a number of vendors offering 'CORBA-compliant' products. Part of the work undertaken within the CSLab has been to assess the relative merits of a number of these offerings and detailed analysis can be found on the CSLab Web pages [1].

6.1 General RPC models

While the three architectures discussed in this paper (CORBA, DCE, DCOM) have their own idiosyncrasies, they also have a common model at their core.

Middleware, as the name implies, is a layer between the operating system and network on the one hand, and the distributed application on the other. The middleware provides the services that allow a distributed application to interact with a collection of possibly heterogeneous

computers, operating systems and networks, as if they were a single system. Figure 5 shows middleware in relation to operating systems, network communications software, and applications software.

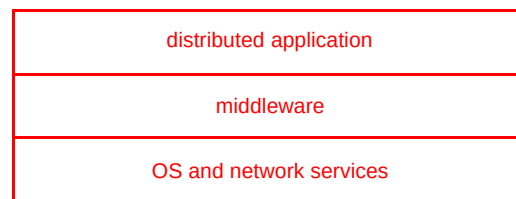


Fig 5 Layering of middleware and related software.

Several technology components work together to implement the middleware layer. Many of these components provide in a distributed environment what an operating system provides in a centralised (single-node) environment.

In general, a middleware's remote procedure call facility consists of both a development tool and a run-time service. The development tool consists of a language (and its compiler) that supports the development of distributed applications following the client/server model. It automatically generates code that transforms procedure calls into network messages. The run-time service implements the network protocols by which the client and server sides of an application communicate. Middleware RPC also includes software for generating unique identifiers, which are useful in identifying service interfaces and other resources.

A directory service is the central repository for information about resources in the distributed system. Typical resources are users, machines, and RPC-based services. The information consists of the name of the resource and its associated attributes. Typical attributes could include a user's home directory, or the location of an RPC-based server.

A fundamental part of any middleware architecture is the RPC support. This support has five principal responsibilities:

- given an object reference from a client, location of the corresponding object implementation (the server) takes place on behalf of the client,
- when the server is located, the RPC mechanism ensures that the server is ready to receive the request,
- the RPC mechanism on the client side accepts the parameters of the method being invoked and marshals the parameter to the network,

- the RPC mechanism on the server side unmarshalls the parameters from the network and delivers them to the server,
- return parameters, if any exist, are marshalled/unmarshalled in the same way.

The process of marshalling and unmarshalling the input/output parameters to/from a format that can be transmitted across a network takes place without any programmer intervention.

The second pillar upon which middleware technologies are built is the concept of the interface definition language. The IDL specification is a standard language used to define the interface used by objects and is responsible for ensuring that data is properly exchanged between dissimilar languages.

The relationship between the middleware and the network and system services is the final generic aspect to consider. As shown in Fig 5, the middleware is layered on top of the local operating system and networking software. The middleware deployed makes certain assumptions about the services provided by the underlying network and operating systems.

In general, middleware is layered over one or more transport level services, such as user datagram protocol (UDP), transmission control protocol (TCP), or ISO TP0-TP4 transport protocols, which are accessed through a transport interface, such as sockets or X/Open Transport Interface (XTI). The middleware assumes that a highly available network physically connects all nodes participating in the distributed environment. The network can be a local area network (LAN), a wide area network (WAN) or a combination of both.

The distributed architecture supports different types of network protocol families; however, in order for distributed systems to communicate with one another, they must have at least one set of network protocols in common.

Middleware also assumes the ability to identify a node with a unique network address, and the ability to identify a process with a network end-point address, e.g. a port or T-selector.

The middleware presumes that certain services are available through the underlying operating system, namely:

- multitasking,
- timers,

- local interprocess communications,
- basic file system operations,
- memory management,
- local security mechanisms (if appropriate),
- threads,
- general system utility functions.

6.2 CORBA

The actual scope of the CORBA standard is wide but in terms of the currently available technology, the CORBA standard can be considered in two parts CORBA and CORBAServices[®] (Fig 6).

6.2.1 Key CORBA concepts

A key component of CORBA is CORBA IDL, a declarative language for specifying remotely invocable (e.g. RPC) interfaces and the data types which are exchanged across them. CORBA IDL has successfully been mapped into a number of programming languages including C, C++, COBOL, Java, Smalltalk and Ada, with a mapping also defined for Microsoft COM.

CORBA IDL has also been mapped into a common data representation (CDR). A general framework for RPC protocols called GIOP (general inter-ORB protocol) has been defined to make use of CDR, and a specific implementation of this over TCP must be supported by all CORBA2.x-compliant products — this standard RPC protocol is called IIOP (Internet inter-ORB protocol). However, in addition to various different specialisations of GIOP, ORBs may utilise any network and RPC protocols to communicate internally and with other middleware products.

In theory, the combination of IDL and IIOP enables an open environment for software components, where components developed using different CORBA-compliant middleware products and different programming languages, running on different platforms, can all interwork via interfaces defined in CORBA IDL. Developer effort in achieving this is minimised, since mapping of data types from one environment to another and network programming are all taken care of by middleware.

CORBA's great potential for systems integration is its greatest selling point and the key driver for its introduction into BT, since it could enable much faster and cheaper evolution of BT OSS, as well as interworking between BT and other organisations, such as customers, OLOs and service providers.

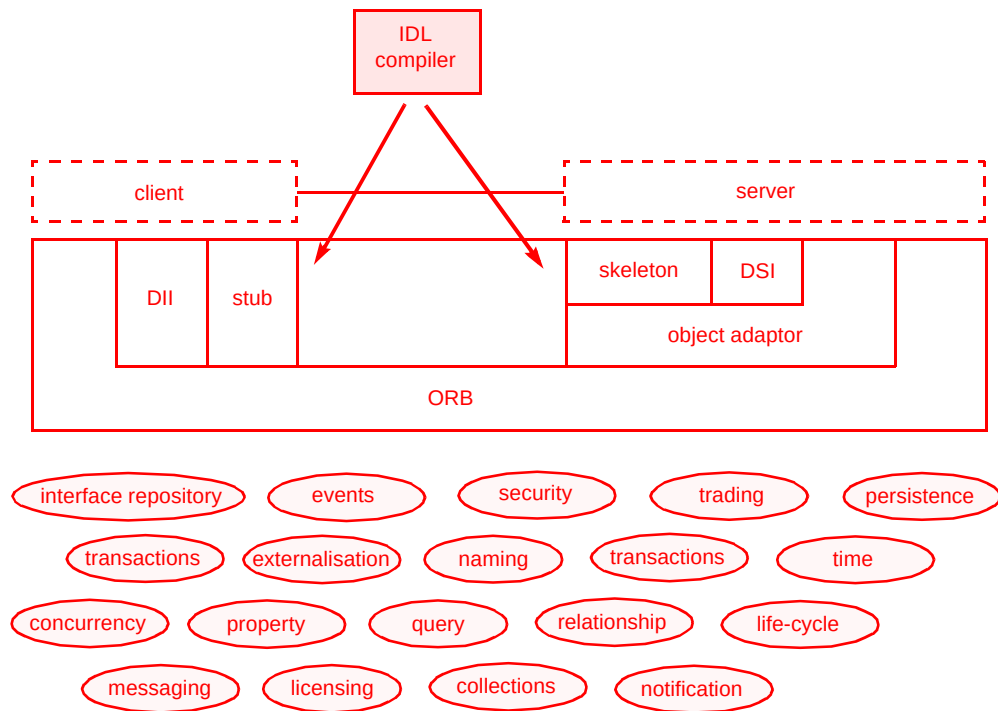


Fig 6 CORBAServices.

In addition to standards for IDL and network protocols, CORBA specifies a large number of APIs for use by application developers. These are defined using IDL and are thus mapped to different programming languages in much the same way as developer-defined IDL interfaces. These APIs are grouped into a number of areas:

- ORBs provide basic functions for obtaining, binding to, invoking and managing references for CORBA objects,
- object adapters provide mediation between network messages to IDL interfaces and in-process calls on objects, object activation and de-activation — CORBA originally specified the basic object adapter (BOA), but the portable object adapter (POA) has now superseded this,
- dynamic invocation interface (DII) supplies facilities for clients to invoke interfaces of types that are discovered at run time rather than part of the original compiling,
- dynamic skeleton interface (DSI) delivers facilities for up-calling server objects dynamically,
- interface repository provides facilities for storing and retrieving information about IDL types on-line — this is useful in conjunction with DII and DSI.

6.2.2 CORBAServices

As well as a core platform to enable interaction between remote software components, based on IIOP and IDL,

CORBA specifies a large number of other services to facilitate development of distributed applications. These services include key aspects such as directory services (Naming Service), security (Security Service) and transactional integrity (OTS).

In addition, although the core of CORBA is based on synchronous RPC mechanisms, there is also an emerging CORBA standard for asynchronous messaging (with guaranteed delivery).

These additional services work over the core ORB platform, but use of a core ORB does not require use of any of these services. Some vendors package services with their core ORB products, while others sell them independently.

However, although the OMG specifications now cover a large number of these services, many of them are not yet widely available as products.

6.2.3 Main benefits of CORBA

The benefits of CORBA are widely believed to be as follows.

- It offers a very good framework for systems integration, with support for interoperability between components developed using different middleware products, different programming languages and different platforms.

- It promotes a consistent infrastructure, since a common set of facilities and services are present in all CORBA-compliant environments. This means that designers can rely upon presence of these services, and thus design applications in a common way in different environments.
- It provides a migration path to new technology. Since CORBA maps well into many technologies, new technologies that appear (such as new programming languages) can be integrated relatively easily. This allows new technology to be deployed alongside heritage systems without loss of investment.
- It enables more flexible product selection, with greater choice and the potential to minimise vendor lock-in. Different technologies can be selected to meet different (especially non-functional) needs, and yet components based on them can interwork. Applications can more easily be ported from one environment to another.

6.3 DCE

Figure 7 shows the DCE architecture and its technology components, along with their relationship to applications, underlying system support, and placeholders for future technologies.

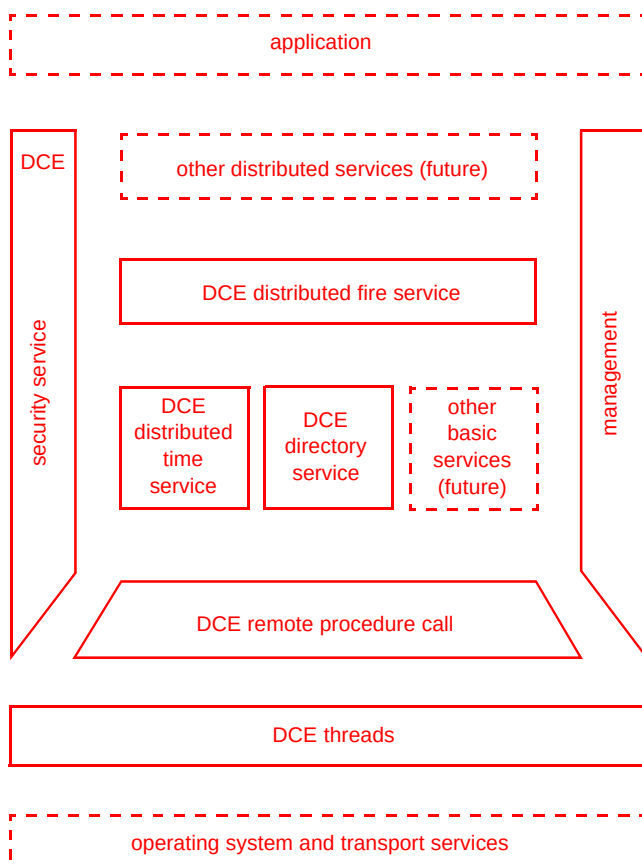


Fig 7 DCE architecture.

6.3.1 DCE technology components

DCE Threads supports the creation, management, and synchronisation of multiple threads of control within a single process. This component is conceptually a part of the operating system layer, the layer below DCE. If the host operating system already supports threads, DCE can use that software and DCE Threads is not necessary.

The DCE Directory Service comprises several parts, the Cell Directory Service (CDS), the Global Directory Service (GDS), the Global Directory Agent (GDA), and a directory service-programming interface. CDS manages a database of information about the resources in a group of machines called a 'DCE cell'. GDS implements an international standard directory service and provides a global namespace that connects the local DCE cells into one world-wide hierarchy. GDA acts as a go-between for cell and global directory services. Both CDS and GDS are accessed using a single directory service application programming interface, the X/Open Directory Service (XDS) advanced programming interface.

The DCE distributed time service (DTS) provides synchronised time on the computers participating in a Distributed Computing Environment (DCE). DTS synchronises a DCE host's time with Co-ordinated Universal Time (UTC), an international time standard.

The DCE Security Service provides secure communications and controlled access to resources in the distributed system. There are four aspects to DCE security — authentication, secure communications, authorisation and auditing. These aspects are implemented by several services and facilities that together constitute the DCE Security Service, including the registry service, the authentication service, the privilege service, the access control list (ACL) facility, the log-in facility and the audit service.

The identity of a DCE user or service is verified, or authenticated, by the authentication service. Communications are protected by the integration of DCE RPC with the security service so that communication over the network can be checked for tampering or encrypted for privacy. Comparing the credentials conferred on a user by the privilege service with the rights to the resource (which are specified in the resource's ACL) controls access to resources. The log-in facility initialises a user's security environment, and the registry service manages the information (such as user accounts) in the DCE security database. Security-relevant events can be monitored through the audit service. 'Code points' can be set in DCE servers to record events that are deemed to be important to the integrity of the system. For example, the log-in facility uses the audit service to record log-ins by DCE users and services.

The DCE distributed file service allows users to access and share files stored on a file server anywhere on the network, without having to know the physical location of the file. Files are part of a single, global namespace, so no matter where in the network a user is, the file can be found by using the same name. DFS achieves high performance, particularly through caching of file system data, so that many users can access files which are located on a given file server without prohibitive amounts of network traffic and resulting delays.

DCE DFS includes a physical file system, the DCE Local File System (LFS), which supports special features that are useful in a distributed environment. They include the ability to replicate data, log file system data (enabling quick recovery after a crash), simplify administration by dividing the file system into easily managed units called 'filesets', and associate ACLs with files and directories.

The 'DCE cell' is a collection of machines, users, and resources managed as a group. A cell has its own security service, CDS, and, optionally, DFS; these services are available cell-wide. The security service for a cell manages the cell's registry, where user account information is kept. Each cell has its own namespace; the cell's CDS manages that namespace and its hierarchy. If DFS is present in the cell, DFS allows remote access to files from anywhere in the cell. Each cell has its own DTS, which keeps the clocks on all of the machines in the cell synchronised.

A cell provides a single security domain. Users log in to accounts in a cell. ACLs identify users and groups in the cell (they can also refer to users and groups in other cells). A cell also provides a single naming domain. Each cell has a name and all objects in the cell share that name.

DCE cells can be connected so that they can communicate with each other. This connection is achieved by means of a global directory service, where the cell's name is registered, thus allowing the cell to contact other cells registered there.

A DCE cell can be configured in many ways, depending on its user's requirements. A cell consists of a network connecting three kinds of nodes — DCE user machines, DCE administrator machines and DCE server machines. DCE user machines are general-purpose DCE machines. They contain software that enables them to act as clients to all of the DCE services. DCE administrator machines contain software that enables a DCE administrator to manage DCE system services remotely.

The DCE server machines are equipped with special software enabling them to provide one or more of the DCE services. Every cell must have at least one each of the following servers in order to function:

- cell directory server,
- security server,
- distributed time server.

6.3.2 Integration of DCE components

Although DCE components are modular with well-defined interfaces, they are also well integrated; the various DCE components each make use of the services of the other components wherever possible. For example, the RPC facility uses the Directory Service to advertise and look up RPC-based servers and their characteristics, it uses the Security Service to ensure message integrity and privacy, and it uses DCE Threads to handle concurrent execution of multiple RPCs. In general, the DCE components shown higher in the DCE architecture (Fig 7) make use of the components shown lower in the architecture.

6.4 DCOM

Microsoft's Distributed COM (DCOM) is an extension of the Component Object Model (COM). COM defines how components and their clients interact. This interaction is defined such that the client and the component can connect without the need for any intermediary system component. COM provides the communication in a transparent fashion; intercepting calls from the client and forwarding them to the component in another process. With DCOM, the local inter-process communication is replaced by a network protocol. Figure 8 shows the overall DCOM architecture. The COM run time provides object-oriented services to clients and components and uses RPC and the security provider to generate standard network packets that conform to the DCOM wire-protocol standard.

6.4.1 Locating objects — activation

At the centre of COM are the mechanisms for establishing connections to components and creating new instances of components. These mechanisms are commonly referred to as 'activation mechanisms'.

One of the most basic requirements of a distributed system is the ability to create components. In the COM world, object classes are named with globally unique identifiers (GUIDs). When GUIDs are used to refer to particular classes of objects, they are called 'class IDs'. These class IDs are nothing more than fairly large integers (128 bits) that provide a collision-free, decentralised namespace for object classes. If a COM programmer wants to create a new object, one of several functions in the COM library is called upon.

The COM libraries look up the appropriate binary code (dynamic-link library or executable) in the system registry, create the object, and return an interface pointer to the caller.

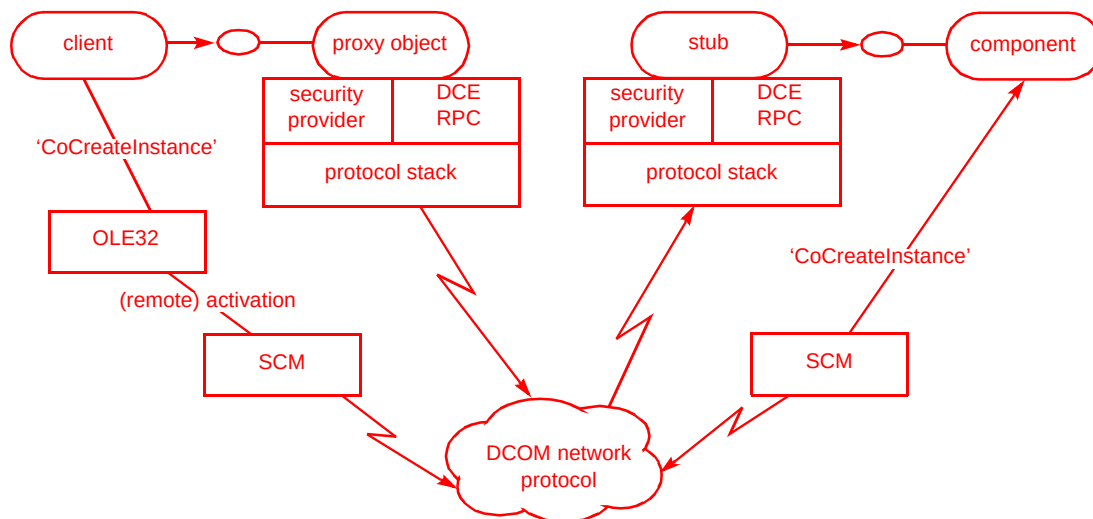


Fig 8 DCOM architecture.

For DCOM, the object creation mechanism in the COM libraries is enhanced to allow object creation on other machines. In order to be able to create a remote object, the COM libraries need to know the network name of the server. Once the server name and the CLSID are known, a portion of the COM libraries, called the Service Control Manager (SCM) on the client machine, connects to the SCM on the server machine and requests creation of the object.

DCOM provides two fundamental mechanisms for allowing clients to indicate the remote server name when an object is created. The remote server name can be indicated:

- as a fixed configuration either in the system registry or in the DCOM Class Store,
- as an explicit parameter to:
 - CoCreateInstanceEx,
 - CoGetInstanceFromFile,
 - CoGetInstanceFromStorage,
 - CoGetClassObject.

6.4.2 Connecting to specific object instances

Often object instances are neither replaceable nor interchangeable; once certain methods have been called, the object has a valuable state that differentiates it from other instances of the same class.

COM defines a naming mechanism for object instances. These instances can be characterised by quite different identifiers (a database name plus a database server name, a URL to an HTML page, etc). The COM instance-naming mechanism makes use of instance names (commonly called 'monikers') which are objects themselves. These naming objects contain the necessary logic to find a currently running instance of the object that they are naming. They

can also re-create and initialise an object instance in case there is no running instance. COM defines a standard interface to these naming objects, 'Imoniker', that can be used to bind to a named object instance. Whatever the moniker object needs to do to connect to the actual object instance is hidden from the client.

6.4.3 Securing DCOM distributed applications

DCOM distinguishes between four fundamental aspects of security:

- access security,
- launch security,
- identity,
- connection policy.

Access security, namely the protection of objects against unauthorised access, is the most obvious security requirement. DCOM provides declarative access control on a per-process level. Existing components can be securely integrated into a distributed application by simply configuring their security policy as appropriate. New components can be developed without explicit security awareness, yet still run as part of a completely secure distributed application.

If an application requires more flexibility, objects can programmatically perform arbitrary validations, on a per-object, per-method, or even per-method parameter basis. Objects might also want to perform different actions depending on the caller, what specific access rights the caller has, or to which user group the caller belongs.

Another related requirement on a distributed infrastructure is to maintain control over who can create

objects. Since all COM objects on a machine are potentially accessible via DCOM, it is critical to prevent unauthorised users from creating instances of these objects. This protection has to be performed without any programmatic involvement of the object itself, since the mere act of launching the server process could be considered a security breach and would open the server to denial-of-service attacks.

The COM libraries thus perform special security validations on object activation. If a new instance of an object is to be created, COM validates that the caller has sufficient privileges to perform this operation. The privilege information is configured in the registry, external to the object.

Since an object performs operations on behalf of arbitrary callers, it is often necessary to limit the capabilities of the object itself. One approach is that of making the object assume the identity of the caller and inherit the caller's privileges.

For applications with large numbers of callers, this approach can impose problems. Managing this problem with the assistance of user groups is a possibility but getting the object itself to run under a dedicated security identity is often simpler.

In all of the security mechanisms described, DCOM ties into the WindowsNT security infrastructure.

6.4.4 Object RPC

The DCOM protocol, known as Object RPC (ORPC), is a set of definitions that extends the standard DCE RPC protocol. It has been designed specifically for the DCOM object-oriented environment and specifies how calls are made across the network and how references to objects are represented and maintained.

Programmers, for the most part, do not have to work at the ORPC level. The Microsoft Interface Definition Language (MIDL) compiler can be used to automatically generate the code that is needed to transfer the data across the network.

7. Java

Java programs have the major selling point of being able to run on a wide variety of computers using a range of operating systems. A Java program will run just as well on a PC running Windows 95 as it will on a Sun Solaris workstation. This is possible because a Java program does not execute directly on the computer. It runs on a standard hypothetical computer, called the 'Java Virtual Machine', which is embedded inside the computer by a program. The Java environment is depicted in Fig 9.

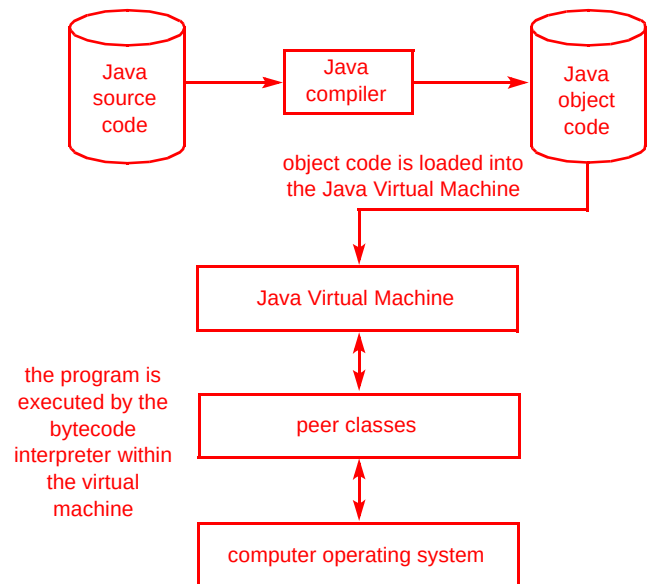


Fig 9 The Java environment.

The Java source code is converted by a Java compiler to a binary program consisting of 'bytecodes', which are machine instructions for the Java Virtual Machine (JVM). When a Java program is executed, a program called the 'Java Interpreter' inspects and deciphers the bytecodes for it, checks it out to ensure that it has not been tampered with and is safe to execute, and then executes the actions that they specify within the Java Virtual Machine. A Java Interpreter can run alone, or it can be part of a Web browser, such as Netscape or Explorer, where it can be invoked automatically to run applets in a Web page.

Because a Java program consists of bytecodes rather than native machine instructions, it is completely insulated from the particular hardware on which it is run. Any computer that has the Java environment implemented will work as well as any other, and because the Java interpreter sits between the program and the physical machine, it can prevent unauthorised actions in the program from being executed.

There is usually a penalty for all this flexibility and protection, and that is in the speed of execution. An interpreted Java program will run at a fraction of the speed of an equivalent program using native machine instructions. If the Java environment supports 'just-in-time (JIT) compilation' of programs, this speed penalty will not be suffered. JIT compilers convert and cache Java programs to native machine instructions, as they are loaded. The programs will, therefore, take longer to load, but once loaded, they execute at maximum speed.

7.1 The Java platform

The Java Language is the entry ramp to the Java Platform. As described above, programs written in the Java

Language and then compiled will run on the Java Platform. The Java Platform has two basic parts:

- Java Virtual Machine,
- Java application programming interface (API).

The Java base platform is the minimum Java Platform that developers can safely assume is present for running Java-powered applets and applications. This platform applies to network computers, desktop computers, and workstations. This platform contains the Java Virtual Machine, but has a minimal set of APIs required to run basic applets and applications. This minimal set is known as the 'Java Applet API', or 'Java Base API'.

The embedded Java Platform is being targeted for consumer devices with fewer resources and more specialised functionality than a network computer. Such devices typically have special constraints such as small memory footprint, no display, or no connection to a network.

The API targeted for this type of platform is called the 'Java Embedded API'. The Java Embedded API is the smallest API a low-function embedded device can have and still run. This platform is still under development and as such has not yet reached the level of a standard.

7.2 Applets and applications

The Java Platform enables developers to create two kinds of program.

- Applets

These are programs that require a browser to run. The <applet> tag is embedded in a Web page and names the program to be run. When that page is accessed by a user, either over the Internet or corporate intranet, the applet automatically downloads from the server and runs on the client machine. Because applets are downloaded, they tend to be designed small or modular, to avoid large download times.

- Applications

These are just like programs in other languages, i.e. they do not require a browser to run — they have no built-in downloading mechanism.

While applets and applications have different means of being invoked, for the most part they have the same access to a wide range of language capabilities. However, an applet requires a network on which to run, while an application does not. Applications have greater freedom in that they have full access to system services. For example, an application, unlike an applet, can have normal read and

write access to files on any disk. Since an applet can potentially be downloaded from an untrusted Web page, it is restricted from having read or write access to any file system except the server from which it came. This constraint will be relaxed when applets can be marked with digital signatures, allowing the end user to be assured that it has been downloaded unaltered from a trusted source. Currently, where local file storage is required, an application is needed.

7.3 Remote method invocation (RMI)

Java remote method invocation (RMI) allows writing of distributed objects using Java. This provides a simple and direct model for distributed objects that can be either Java objects or simply wrappers around existing APIs. For example, RMI can connect to existing legacy systems using Java Native Interface (JNI) or connect to existing databases using Java Database Connectivity (JDBC).

At the most basic level RMI is Java's RPC mechanism with the exception of having several advantages over the traditional RPC systems. The main advantages of RMI are:

- object-oriented — RMI can pass full objects as arguments and return values as well as the predefined data types,
- mobile behaviour — this allows moving behaviours (class implementations) from client to server and server to client,
- safe and secure — RMI uses built-in Java security and security manager to protect systems from hostile applets,
- connection to legacy systems — RMI allows connection to legacy systems through the use of JNI and JDBC,
- distributed garbage collection — this is used to collect and remove remote objects that are no longer referenced.

7.3.1 Distributed object applications

RMI applications often comprise two separate programs, namely a 'server' and a 'client', where a typical server creates a number of remote objects, makes references to them accessible, and waits for clients to invoke methods on those objects. A typical client gets a remote reference to one or many objects and then invokes methods on them.

RMI provides the mechanism by which the server and client communicate and pass information to each other. To offer such facilities RMI provides the following:

- locating the remote object — applications can either register their remote objects with RMI's simple naming service or pass and return remote object references,
- communication with remote objects — RMI hides this complexity from the user and handles communications internally,
- loading class bytecodes — RMI provides necessary mechanisms for loading an object's code as well as its data.

Figure 10 illustrates an RMI distributed application that uses the registry to obtain references to remote objects.

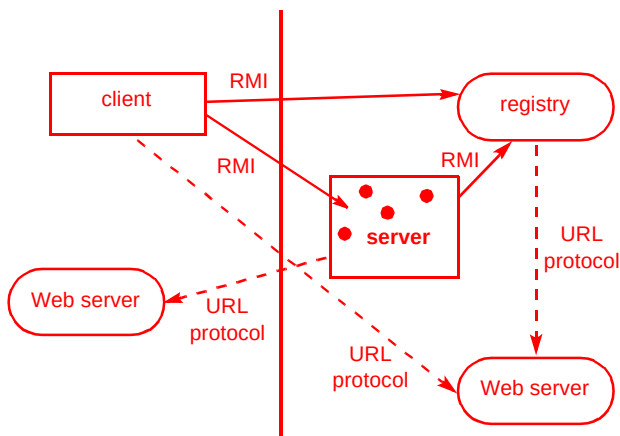


Fig 10 RMI distributed application.

The server registers some of its method with the registry. The client looks up the remote object by its name in the registry and then invokes methods on it. Figure 10 also shows that the RMI system uses an existing Web server to load class bytecodes, from server to client and from client to server, for objects when needed. RMI can load class bytecodes using any URL protocol (e.g. HTTP, FTP, file) supported by the Java system.

7.3.2 Stubs and skeletons

RMI uses a standard mechanism (similar to one used in RPC systems) for communication with remote objects namely 'stubs' and 'skeletons'. A stub for a remote object acts as a client's local representative or proxy for the remote object. In the remote virtual machine, each remote object may have a corresponding skeleton (in JDK1.2-only environments, the skeleton is not required). This is shown in Fig 11.

The caller invokes a method on the local stub that is responsible for carrying out the method call on the remote object. In RMI a stub for the remote object implements the same set of remote interfaces that a remote object

implements. When a stub's method is invoked, it does the following:

- initiates a connection with the remote VM,
- writes and transmits the parameters to the remote VM,
- waits for the result of the method invocation,
- reads the return value or exception,
- returns the value to the caller.

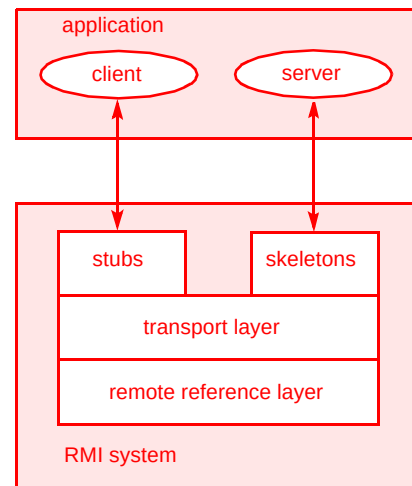


Fig 11 Java stubs and skeletons.

The stub simply hides the serialisation of parameters and the network level communications.

In the remote VM the skeleton is responsible for despatching the call to the actual remote object implementation. When the skeleton receives an incoming method invocation it does the following:

- reads the parameters for the remote method,
- invokes the method on the actual remote object,
- writes and transmits the result or exception to the caller.

In JDK1.2, an additional stub protocol was introduced that eliminates the need for the skeleton.

7.3.3 RMI transport protocol

The wire format for the RMI is represented by a 'stream'. The input and output streams used by RMI are paired. An output stream simply maps to the output of a socket and the input stream is paired with the corresponding socket's input stream.

The following three protocols are supported for the output stream:

- StreamProtocol,
- SingleOpProtocol,
- MultiplexProtocol,

SingleOpProtocol is used for the invocation embedded in HTTP requests where interaction beyond a single request and response is not possible. However, StreamProtocol and MultiplexProtocol require the server to respond with an acknowledgement and a host name/port number that the client can use for communication with the server.

7.4 Enterprise JavaBeans

Enterprise JavaBeans (EJB) defines a component model for the development and deployment of Java applications based on a multi-tier, distributed object architecture. Java currently has a component model (JavaBeans). Enterprise JavaBeans extends the JavaBeans component model to support server components.

The main goals of EJB can be summarised as:

- EJB will be the standard component architecture for building distributed object-oriented business applications in Java — EJB will make it possible to build distributed applications by combining components developed using tools from different vendors,
- the EJB architecture will be compatible with other Java programming language APIs,
- the EJB architecture will provide interoperability between Enterprise JavaBeans and non-Java programming language applications,
- the EJB architecture will be compatible with CORBA.

Enterprise JavaBeans describes, among other things, a Java component model; however, this model is not the same as JavaBeans. With JavaBeans the emphasis is on allowing developers to visually manipulate components in a builder tool. To that end, the JavaBeans specification describes in detail the APIs and semantics for inter-component event registration and delivery, recognition and utilisation of properties, customisation, and persistence.

The emphasis for EJB is to detail a model for a services framework into which Java components can be portably deployed. Consequently, there is no mention of events, since enterprise beans do not typically send or receive events. Nor is there a mention of properties. Customisation does exist, but is not performed at development time using

properties, but at run time (deployment time, actually) using a deployment descriptor.

JavaBeans and Enterprise JavaBeans are both specifications for component models, with one addressing issues of application assembly in a builder tool, while the other details a services framework into which components can be deployed.

7.4.1 Services framework

As Enterprise JavaBeans is not a product (but a specification for a Java server-side services framework), it allows vendors to create EJB server implementations. The benefit to application developers is that they can focus on writing the business logic necessary to support their application without worrying about implementing the surrounding framework.

The specification details certain minimal services such as transactions, security, and naming. The vendors must follow these specifications to ensure that an EJB can always rely on certain required services.

7.4.2 Enterprise JavaBeans architecture

The EJB server is the high-level process or application that manages EJB containers, along with providing access to system services. The EJB server may also provide vendor-specific features, e.g. optimised database access interfaces, and availability of additional services such as CORBAServices, support for SSL 3.0, etc.

An EJB server is required to provide availability of a JNDI-accessible naming service and a transaction service, such as Object Transaction Services (OTS). Figure 12 shows a typical EJB server.

Some examples of EJB servers could be:

- database servers,
- application servers,
- middleware servers.

The EJB container is an abstraction that manages one or more EJB classes and/or instances. It makes required services available to the EJB classes through an interface defined in the specification.

The home interface lists the available factory methods for locating, creating, and removing instances of EJB classes. The home object is the implementation of the home interface. The developer of the EJB class must also define the home interface. The container vendor should provide the

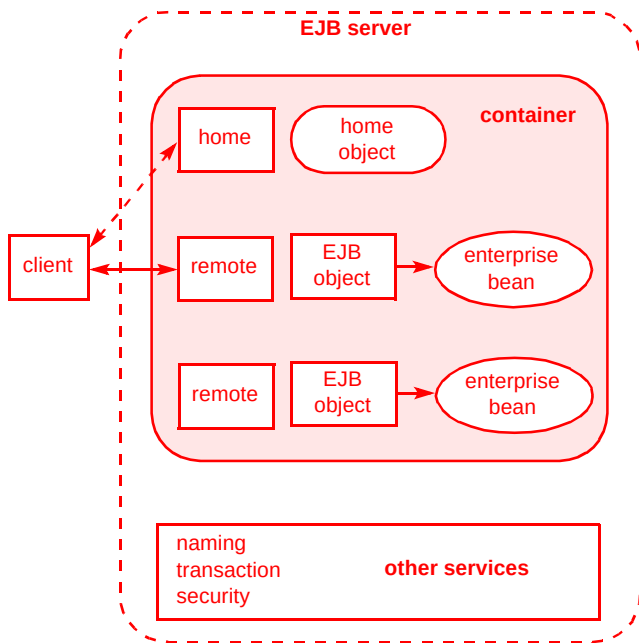


Fig 12 An EJB server.

means to generate the home object implementation from the home interface.

The remote interface lists the business methods in the EJB class. The EJBObject implements the remote interface, and is the object that the client must use to access the business methods of the EJB instance. The EJB class developer defines the remote interface, and the container vendor should provide the means to generate the corresponding EJBObject.

The client can be written in Java and use Java RMI to access the home object and EJBObject, or it may be written in another language using CORBA/IIOP, providing the required server-side components are deployed in a manner that makes them accessible through CORBA interfaces.

7.4.3 Scenario

StockTrader (a fictitious company), a large brokerage house, is using Enterprise JavaBeans to implement an application system to manage stock fund accounts. StockTrader wants to provide clients with a self-service electronic trading system. To effectively support broker and client users, the application is implemented using a thin-client, distributed object architecture. The architecture of the application is depicted in Fig 13.

Rather than building the entire application from scratch, StockTrader purchased a stock fund account management system, implemented in Enterprise JavaBeans, from an application software vendor. The application vendor developed the application using an Enterprise JavaBeans server provided by a database vendor.

Although the application provides the functions required to manage client accounts and organise portfolios, it does not use up-to-the-minute stock prices to calculate the current account value. The application also does not support on-line trading.

StockTrader customised the stock component to make it retrieve the most recent stock price from a live data feed. StockTrader used a stock feed Enterprise JavaBeans from

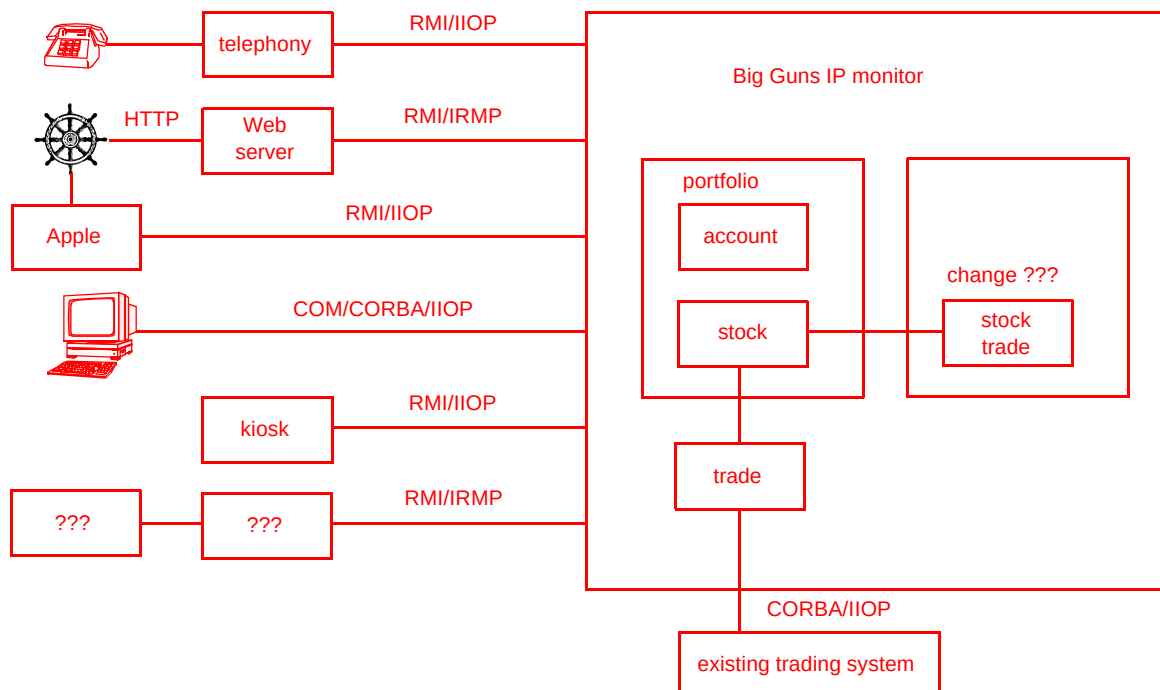


Fig 13 'StockTrader' architecture.

another vendor. This component was developed using a public domain Enterprise JavaBeans server.

StockTrader decided to implement the on-line trading function by encapsulating the existing trading system and integrating it with the portfolio system using CORBA.

8. CSLab evaluation activities

The CSLab forms part of the systems evolution theme within the Systems and Software Strategy (S&SS) unit in the Ipswich Software Engineering Centre (ISEC).

The CSLab is a corporately funded project which is looking at new middleware technologies in order to evaluate them as alternatives to, and their interoperability with, BT's existing middleware, the Distributed Computing Environment (DCE).

The resolution of the problem of how to best exploit the new emerging middleware technologies is the main work activity for the CSLab. Within the CSLab there are two main strands of expertise and study and these are detailed below.

8.1 CORBA for the enterprise

The evaluation work described in this section has been carried out in association with a project called the 'IT Infrastructure Programme'. Similar work has also been carried out in a number of other projects. This section does not describe all of these, but contact with other areas supports similar general conclusions.

Since the spring of 1997 a range of CORBA evaluation tasks have been conducted within the CSLab to help de-risk CORBA technology for mainstream deployment within BT, and ascertain where and when CORBA technology should be used, and for what. This work was aimed at mid-tier mission-critical OSS systems, but is also applicable to other areas.

Experience has been gained in the CSLab with the following:

- portability between CORBA products and porting from DCE to CORBA,
- interoperability between CORBA products,
- management of CORBA-based systems,
- performance and scalability of CORBA products, and in comparison with DCE,
- a range of CORBA products including Orbix and Visibroker, with both C++ and Java,
- CORBAServices — naming, events and OTS.

Detailed reports on these work areas can be obtained from the CSLab Web pages [1].

In addition, evaluation work has been done with a number of other products in association with the Enterprise Middleware initiative. These include Transarc Encina, BEA M3, IBM Component Broker and Oracle Application Server.

It is important to recall that CORBA is the only vendor-neutral standard for heterogeneous distributed software systems — other technologies in this space, such as DCOM and DCE, are each based around a specific implementation or code-base.

The CORBA specifications have been maturing gradually since the first version was issued in 1991. Although still far from complete, the core areas of CORBA have become much more thoroughly defined over this time, and there is now some hope that they will remain relatively stable.

The CORBA specifications suffer from the same flaws and problems as other paper-based standards generated by committees:

- they are ambiguous in places,
- because they do not specify all aspects of middleware, CORBA-compliance is no guarantee that a product is fit for use,
- they enshrine the 'state of the art' in the distributed systems industry at the point when they were defined (e.g. IDL in 1991) and may thus be rendered obsolete by advances in technology.

Despite these problems, in general the CORBA specifications are well thought out and meet their goals of scalability and heterogeneity well. CORBA is thus the most credible vehicle to support heterogeneity in distributed systems. Currently, although the CORBA standard promises to be a very useful vehicle for BT to reduce costs and time-scales for systems integration in the longer term, no CORBA products have yet been identified within BT as suitably mature for development of mission-critical systems.

However, all of the major middleware vendors except Microsoft have now adopted a strategy that incorporates CORBA, and even Microsoft have made alliances to promote interoperability between CORBA and COM. It seems only a matter of time before CORBA becomes the basis for most mainstream middleware technologies.

8.1.1 Status of CORBA products

There are a large number of CORBA-compliant middleware products in existence, including:

- IONA Orbix,
- Inprise Visibroker,
- Sun NEO (now obsolete),
- ICL DAIS,
- BEA M3,
- IBM ComponentBroker,
- Oracle Application Server,
- Fujitsu ObjectDirector,
- Expersoft CORBAplus.

Many of these have not been evaluated within BT, and consequently it is impossible to make definitive statements about their collective maturity.

Within the CSLab, paper-based evaluation work has been done with BEA M3, IBM Component Broker and Oracle Application Server. In these cases, the products look promising, and meet many of BT's middleware requirements, but have not yet matured sufficiently for mission-critical use.

Hands-on experience has been gained with IONA Orbix and Inprise Visibroker.

These products can generally be made to work, with sufficient effort. However, there are likely to be problems that must be solved in order to deploy stable systems based on them, such as working around bugs or developing extra infrastructural software. Specific developments should be carefully de-risked through initial feasibility work, and the amount of time required to solve these problems should not be underestimated.

8.1.2 Guidelines for selecting ORBs

Experience with distributed computing is essential to make non-trivial usage of CORBA successful, both in terms of development processes and practices, and in terms of systems design.

Experience and guidelines that are concerned with distributed computing in general rather than CORBA in particular are not described here, but can be sought from CSLab [1].

For projects contemplating use of CORBA-compliant products, the following suggestions should be considered.

- Feasibility assessment

Engineers should not assume that CORBA technology will work 'out of the box' and should plan in enough time to de-risk a project before committing, or else identify a clear fallback strategy in case of failure.

- Use of core features

Avoid reliance on peripheral or little-used aspects of CORBA, and use only core features where possible.

- Interworking

Avoid non-essential or complex interoperation between different CORBA products, since this is often error-prone.

- Internet deployment

Deployment of CORBA technology on the Internet, i.e. direct IIOP communication between an Internet client and an intranet server, can be difficult to achieve, due mainly to issues of security and scale.

- Avoidance of proprietary features

Vendors often do not distinguish between CORBA APIs and proprietary ones — use standard APIs where possible to minimise vendor lock-in.

- Availability of services

Check availability of any CORBAServices, such as Security and Transactions, which may be required for later phases of a project, before selecting a core CORBA product.

- Performance check

As performance of ORB products varies greatly, if performance requirements are stringent, verify that a product meets them before selection. Performance figures are widely available.

- Systems management requirements

Since the CORBA specifications largely ignore systems management issues, great care should be taken to address systems management requirements at an early stage, and not to expect the ORB to solve them out of the box.

8.2 Java for the Enterprise

The 'Java for the Enterprise' evaluation activity within the CSLab has, in the same vein as the CORBA evaluation activity, been carried out in association with the IT Infrastructure Programme.

This activity was started in April 1997, and has as its objectives:

- to evaluate benefits and impact of Java to BT's OSS,
- to make recommendations on this technology,
- to test non-functional aspects of Java-based systems, e.g. performance, scalability,
- to develop skills and infrastructure within the CSLab to enable it to act as a reference site within BT for distributed systems based on Java technology.

The work undertaken within the CSLab project has considered both client- and server-side Java. The work on the client-side aspects has concentrated on:

- GUI,
- Applets,
- ORB interoperability,
- software distribution;

while the server-side studies have looked at:

- insertion of data into Oracle databases,
- performance (throughput),
- scalability.

The results from the various activities can be found on the CSLab Web pages [1].

BT's Java Forum [2] has produced the following policy statement on the use of Java within BT.

'The Java technology set is not approved for general software development use within BT. This does not mean that Java is being ignored, or discounted, as a tool for possible future use. A number of controlled evaluation activities are in progress to ensure that BT fully understands the strengths and weaknesses of Java. This will enable BT, at the appropriate time, to leverage the maximum potential from the use of Java.'

The CSLab team has played an active role in the formulation of this policy and the issues list drawn up by the BT Java Forum. The recommendations to the Forum were

made as a result of the experiences gained during the early evaluation activities.

As a result of recent studies, however, the CSLab team are confident enough in the technology to recommend that, while Java should not be used in any mission-critical applications, it could have limited use in non-mission-critical applications where the cost of failure of the system is lower.

9. Conclusions

The world of Enterprise Middleware is rapidly evolving into one of critical importance to large corporations such as BT. To achieve maximum benefit from this technology, a consistent approach across all OSS developments must be ensured.

This paper has detailed some of the choices of middleware technologies currently available, and details some of the key issues that development teams should consider before making their choice of technology.

The number of vendors on the market is increasing (Microsoft, IBM, BEA, Iona, etc) and each vendor has targeted its products at different niche markets.

Given the wide-ranging capabilities of the different offerings on the market, it is very difficult to make a firm recommendation on a product or technology that is applicable across all developments. BT's Enterprise Middleware Initiative is seeking to determine a common approach across the whole range of OSS.

References

- 1 CSLab Web Pages — <http://www.srd.bt.co.uk/themes/SE/www/cslab/cslab.htm>
- 2 BT's Java Forum Web Pages — <http://narn.ilford.bt.co.uk/java>



Steve Benns leads the Client Server Lab team looking into distributed computing technologies and architectures. The team has spent the last two years looking at the role that Java, CORBA and DCOM can play in BT's next generation Operational Support Systems.

He joined BT in 1988 after graduating from the Royal Military College of Science, and spent the first nine years of his career in the Network Transport Engineering Centre where he led a team developing software planning tools for fixed and mobile radio in the UK and mainland Europe.



Ben Crawford has worked in the Client Server Lab for two years, evaluating middleware and developing advanced software. His main areas of expertise are distributed systems architectures, middleware technologies, CORBA, and systems integration.

Prior to this, his time within BT was spent working on design and development of operational support systems (OSS) such as TDS and Concert, and research into new architectures for OSS.

He also spent two years working on the ANSA project, a collaborative research project which was very influential within the middleware industry, and was funded by a consortium which included BT.



Mohammed Touman holds a BEng (1st class Hons) in Digital Systems Engineering and is currently finishing his PhD in Management of Broadband ATM Networks at Durham University. He has worked in various roles (researcher, developer, designer, architect) for several universities and companies (Durham University, QSP plc, British Gas, IBM UK) before joining BT in 1995. He has a broad range of experience in client/server, distributed systems and network computing and has extensive experience of using Internet technologies, Java and emerging technologies, such as Jini, in implementing such systems. He has worked as the lead developer for graphical front-ends (particularly 3-D and virtual reality) for several projects, and has a keen interest in future human/computer interaction and intelligent systems. He has been involved in several major development and evaluation studies including MapTool (a GIS front-end to some of the MCSS functionalities), ServiceView eCommerce and evaluation of Java GUI classes for future OSS.